JavaScript is disabled on your browser.

java.lang

Annotation Type SafeVarargs

```
@Documented
 @Retention(value=RUNTIME)
 @Target(value={CONSTRUCTOR,METHOD})
public @interface SafeVarargs
```

A programmer assertion that the body of the annotated method or constructor does not perform potentially unsafe operations on its varargs parameter. Applying this annotation to a method or constructor suppresses unchecked warnings about a *non-reifiable* variable arity (vararg) type and suppresses unchecked warnings about parameterized array creation at call sites.

In addition to the usage restrictions imposed by its `@Target` meta-annotation, compilers are required to implement additional usage restrictions on this annotation type; it is a compile-time error if a method or constructor declaration is annotated with a `@SafeVarargs` annotation, and either:

- the declaration is a fixed arity method or constructor
- the declaration is a variable arity method that is neither `static` nor `final`.

Compilers are encouraged to issue warnings when this annotation type is applied to a method or constructor declaration where:

- The variable arity parameter has a reifiable element type, which includes primitive types, `Object`, and `String`. (The unchecked warnings this annotation type suppresses already do not occur for a reifiable element type.)
- The body of the method or constructor declaration performs potentially unsafe operations, such as an assignment to an element of the variable arity parameter's array that generates an unchecked warning. Some unsafe operations
  do not trigger an unchecked warning. For example, the aliasing in

```
@SafeVarargs // Not actually safe!
static void m(List<String>... stringLists) {
```

```
    Object[] array = stringLists;
    List<Integer> tmpList = Arrays.asList(42);
    array[0] = tmpList; // Semantically invalid, but compiles without warni
    String s = stringLists[0].get(0); // Oh no, ClassCastException at runti
}
```

leads to a `ClassCastException` at runtime.

Future versions of the platform may mandate compiler errors for such unsafe operations.

Since:

1.7

See *The Java™ Language Specification*:

4.7 Reifiable Types, 8.4.1 Formal Parameters, 9.6.3.7 @SafeVarargs

Submit a bug or feature
For further API reference and developer documentation, see Java SE Documentation. That documentation contains more detailed, developer-targeted descriptions, with conceptual overviews, definitions of terms, workarounds, and working code examples.
Copyright © 1993, 2026, Oracle and/or its affiliates. All rights reserved. Use is subject to license terms.
Also see the documentation redistribution policy.

Scripting on this page tracks web page traffic, but does not change the content in any way.